

6. **USB client port:** While uploading the code on the board, you need to connect Galileo to the computer using this port. Make sure the external power supply is connected before connecting the board to the computer.
7. **USB host port:** You can connect different USB computer peripherals, like a webcam and sound devices, to this port.
8. **Serial port:** This is meant for interacting with Galileo's Linux OS via a text based command line environment.
9. **Power input:** Connect your power supply to this port, and always do so before uploading the code to the board.
10. **Ethernet port:** You can connect to a wired network using this port.
11. **Clock battery power:** A 3V coin cell battery can be used to keep track of the date and time, even when the power supply is not supplied to the board.
12. **Reboot button:** This button will reboot the program as well as the Linux OS.
13. **Reset button:** This is similar to the Arduino 'Reset' button and will reset the code.
14. **JTAG header:** This is generally used by the more knowledgeable hobbyists to test and debug boards.
15. **IOREF jumper:** This lets you change the logic voltage level of the board from 5V to 3.3V for compatibility with 3.3V shields and components.
16. **VIN jumper:** Pulling this jumper out will disconnect the VIN pin from Galileo's 5V regulator.
17. **I2C jumper:** This lets you change the I2C address of a couple of on-board components.

Intel Galileo and Arduino Uno

If you have used Arduino Uno before, you won't be able to find many differences between it and Intel Galileo, except for the size. In fact, Galileo is a cross between an Arduino Uno and a low-end computer. Along with the larger form factor, it has a more powerful processor, more memory and more data storage space. Additionally, it has the ability to connect computer accessories through a USB port and a Mini PCI Express connector at the bottom. Typical Arduino boards have firmware called *boot loaders* which help you upload and run the code on the board. On the other hand, Galileo firmware is capable of doing much more because it is a version of Linux. It keeps track of files, the date and time of the day, and helps share the board's various resources between multiple programs running at the same time. Although it doesn't have a screen or GUI, it has much of the functionality of an OS. Using your Arduino code, you will be able to access these functionalities,

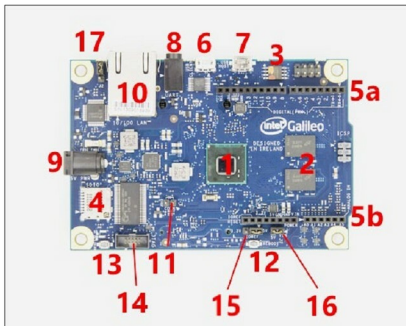


Figure 1: Various component of Intel Galileo

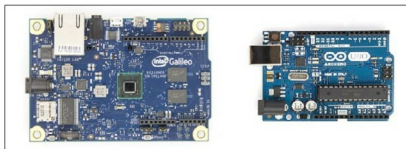


Figure 2: Snapshots of Intel Galileo (L) and Arduino Uno (R)

Applications of the Intel Galileo board

- All the Arduino projects
- An app to run a *Node.js* server
- An app to control a robot
- Gesture based remote control software
- UAV projects
- ...and much more.

giving you much more capabilities than you'd have with a typical Arduino. For example, if you want your project to take a picture from a webcam and email it, it's something that would be difficult to do with only Arduino code. With the power of Linux, however, this could be done much more easily.

The next article will cover how you can use Galileo as an Arduino board and some cool projects. 

By: Samarth Shah

The author is an engineer by profession and maker at heart. He leads maker activities at Pure Makers (<http://www.meetup.com/Pure-Makers/>), and has authored a book titled "Learning Raspberry Pi". He can be reached at samarth@outlook.com